MODULE I

Introduction to C Programming

C is a procedural programming language initially developed by Dennis Ritchie at Bell Laboratories in 1972. It was developed along with the UNIX operating system. The syntax of many programming languages including Java, PHP, JavaScript is primarily based on the C language.

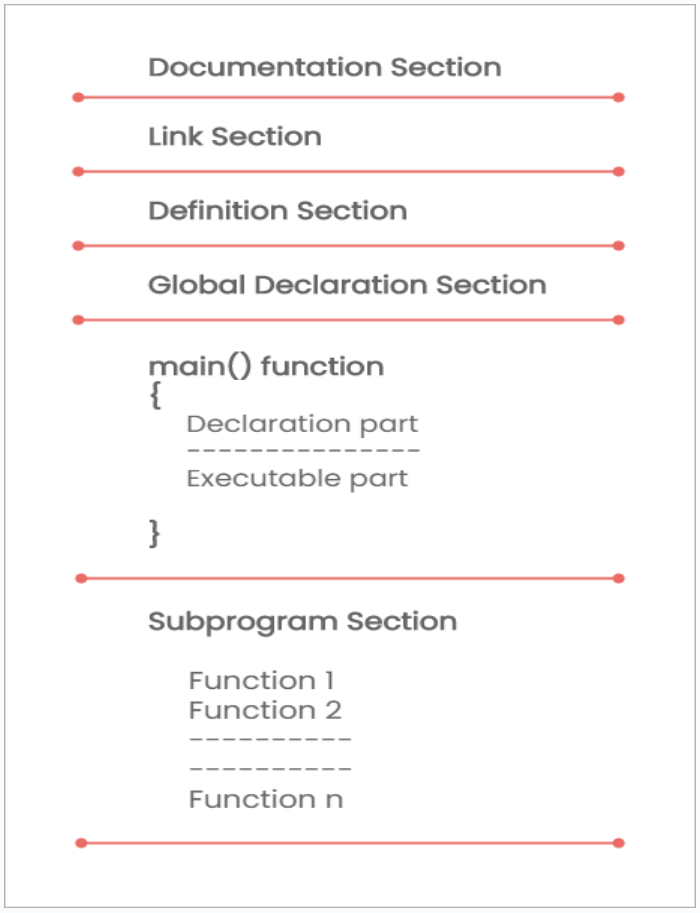
Features of C Programming Language

The growing popularity of C is most likely due to its many desirable characteristics. The following are the important features of the C programming language.

- Robust language – Well suitable for writing any complex programs.
- Fast and efficient programs – Because of variety of data types and operators.
- Rich set of inbuilt functions.
- Highly portable.
- Well suited for structured programming.
- Ability to extend itself.

Basic Structure of C Programs

A C program can be thought of as a collection of blocks known as functions. A function can be defined as a subroutine that contains one or more statements that perform a specific task. A C program may contain one or more sections as follows.



Structure of C Program

The **document section** of the program is made up of a set of comment lines containing details like the name of the program, author, etc.

The **link section** contains instructions to the compiler on how to link functions from the system library.

All symbolic constants are defined in the **definition section**.

Variables that are used in multiple functions are known as global variables. These variables are declared in the **global variables section**.

Every C program should have a **main()** function section. It is a special function used by the C to tell the computer where the program starts. The main() section is divided into two parts: the **declaration part** and the **executable part**. The declaration section declares all variables. The executable section contains at least one statement.

These two parts must be written inside the opening and closing braces. All statements in the declaration and executable sections are followed by a semicolon (;).

All user-defined functions that are called in the main function are written in the **subprogram section**.

Except for the **main** function section, all sections may be skipped when they are not required.

Hello World Program in C

The following program in C prints the message

“Hello World !” on the screen as below:

```
#include<stdio.h>

main()

{

printf("Hello World!");

}
```

Executing a C Program

Executing a program written in C involves the following steps:

1. Creating the program.
2. Compiling the program.
3. Linking the program with functions that are needed from the C library.
4. Executing the program.

ELEMENTS OF C LANGUAGE

Character Set

The set of alphabets, digits and special characters that are valid in the C language is called the **character set**.

Alphabets

Both lowercase (a – z) and uppercase (A – Z) alphabets are accepted by C.

Digits

C accepts all digits from 0 to 9.

Special Characters

C supports the usage of the following special characters.

,	–	$	]	&	^	|
<	(	:	#	{	!	–
>	)	%	?	}	*	\
.	;	[	‘	“	/	~
+						

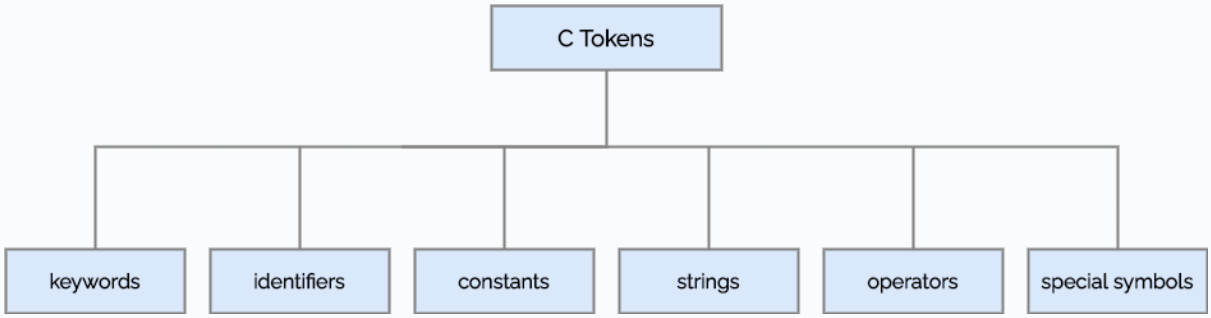
White spaces

Blank spaces, horizontal tab, carriage return, newline and form feed characters are allowed in C. White spaces may be used to

separate words, but they are not permitted between keywords and identifiers.

C tokens

The smallest individual units in a C program is known as C tokens. C has six types of tokens as shown in the figure.



Keywords

Predefined reserved words with fixed meanings are called keywords. Since keywords are the building blocks for program statements, they cannot be used as identifiers. Since C is case sensitive, all keywords must be written in lowercase. The list of all keywords available in C is given below:

- **auto**
- **break**
- **case**
- **char**
- const
- **continue**
- **default**
- **do**
- **double**
- **else**
- enum

- **extern**
- **for**
- **float**
- **if**
- **goto**
- **int**
- **long**
- **register**
- **return**
- **short**
- **static**
- sizeof
- signed
- struct
- **switch**
- typedef
- union
- **void**
- unsigned
- volatile
- **while**

Identifiers

The names of variables, functions, and arrays are referred to as identifiers. These are user-defined names consisting of a sequence of letters and digits. Identifiers must be unique.

The rules for naming identifiers in C

- Must consist of only letters, digits or underscore.
- Both uppercase and lowercase letters are permitted.
- The first character must be a letter of the alphabet or underscore.
- Only first 31 characters are significant.
- Cannot use a keyword.
- Must not contain white space.

For example:

```
int age;
float mark;
```

Here age and mark are identifiers.

Variables

A **variable** is a data name that can be used to store data. A variable may take different values at different times during the execution of a program. A variable name can be meaningfully chosen by the programmer, subject to the conditions listed below:

- Variable names can be made up of alphabets, digits, and the underscore character.
- They must begin with an alphabet. Some systems allow underscore as the initial character.
- Uppercase and lowercase letters are significant. That is, the variable Mark is not the same as mark or MARK.
- It should not be a keyword.
- White space is not allowed.

Some examples of valid variable names includes
Value, score, mark1, phone _number, Counter_1, etc.

Invalid examples include 123, (area), 22nd, char, etc.

Declaration of Variables

Declaration tells the compiler what the variable name is and what type of data the variable will hold. In C, variables can be declared using the following syntax:

```
datatype variable1;
```

Multiple variables can be declared by separating them with commas as follows.

```
datatype variable1, variable2, variable3;
```

For example,

```
int x;

float a, b, c;
```

Here

int and float are the keywords used to represent integer and real numbers respectively.

Initializing variables

Assigning value to a variable once declared is called initialization of a variable. In C, the variable declaration and initialization together takes the following form:

```
int x = 5;
```

The following ways are also acceptable.

```
int a, b=10, c = 5;

int x, y, z;

x = 10;

y = z = 5;
```

Here

b = 10, c = 5, x =10, y= 5, z = 5.

Constants

Fixed values that do not change throughout program execution are called **constants**. The keyword const can be used to define a constant as follows.

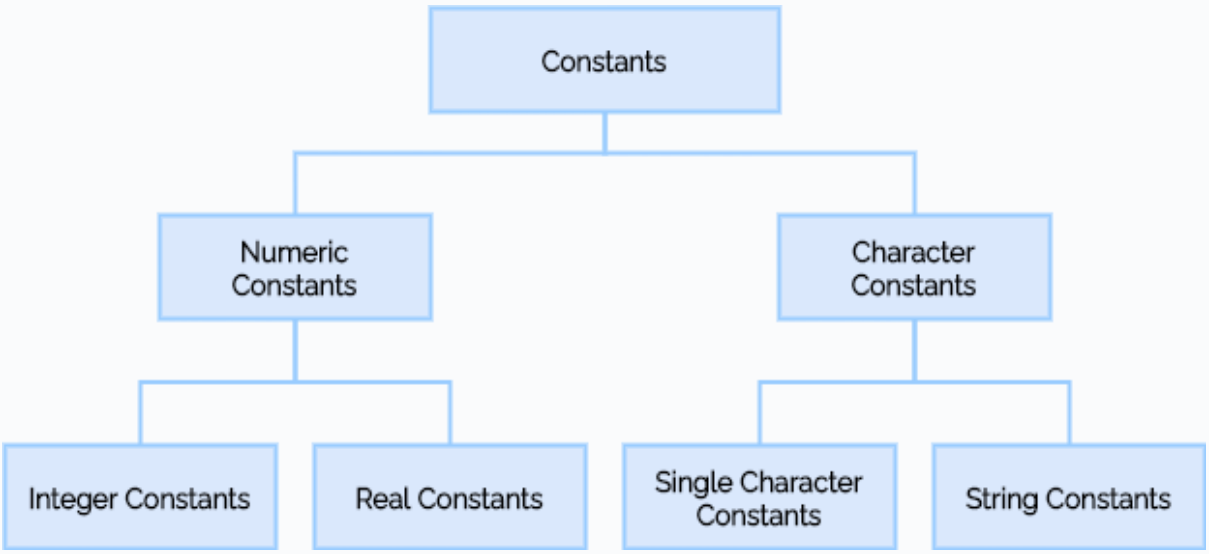
```
const datatype name = value;
```

For example

```
const double pi = 3.14;
```

will define a constant named 'pi' of type 'double' having the value '3.14'. The value of pi cannot be changed throughout the program.

Constants can also be defined using the #define preprocessor directive. You can learn about the symbolic constants in this tutorial.C supports the following types of constants.



Types of constants in C

Integer Constants

A sequence of digits is referred to as an integer constant. Spaces, commas, and non-digit characters are not permitted between digits. The integer constants are of three types.

- Decimal integer – Consist of a series of digits from **0** to **9**, followed by an optional – or + symbol.
- Octal integer – Made up of any combination of digits from **0** to **7**, with a leading 0.
- Hexadecimal integer – Series of digits preceded by **0x** or **0X**.

Example:
Decimal Integer: 120 -450 1 654321 +56

Octal Integer: 031 0 0321 04320

Hexadecimal: 0x2F 0X1A2 0xD10 0x

Real Constants

Numbers containing fractional parts are called real constants. Examples of real constants include 452.5 -.74 +321.2 etc.

Single Character Constants

A single character enclosed within a pair of single quote marks is called a **single character constant**. Examples of single character constants include 'F' 'c' '3'.

String Constants

A sequence of characters enclosed in double quotes is called a string constant. Letters, numerals, special characters, and blank spaces can all be used as characters. Some examples are "Hello" "Good Morning" "1996" "what?" etc.

Backslash Character Constants

C includes various special backslash character constants that can be used in output functions. They are known as escape sequences. The following table shows the list of backslash character constants in C.

Constant	Meaning
\a	Bell
\b	Back space
\f	Form feed
\n	New line
\r	Carriage return
\t	Horizontal tab
\v	Vertical tab
\’	Single quote
\”	Double quote
\?	Question mark

Constant	Meaning
\\	Back slash
\0	Null

C Data Types

Data types are used to determine the size and type of data stored in a variable. For example,

```
int mark;
```

Here mark is a variable of type int. That is mark can store integer values.

C supports the following classes of data types:

- Primary data types.
- Derived data types.
- User-defined data types.

1.Primary Data Types

C supports five primary (fundamental) datatypes namely **integer**(**int**), **character**(char), **floating-point** (float), **double-precision floating-point**(double) and **void**(**void**). Many of them also include additional data types, such as long int and long double.

The following table contains various data types along with their size, range of values and format specifier.

Type	Size (bytes)	Range of values	Format specifier
int	2	-32,768 to 32767	%d
char	1	-128 to 127	%c
float	4	3.4E -38 to 3.4E +38	%f
double	8	1.7E -308 to 1.7E +308	%lf
short int	1	-128 to 127	%hd
unsigned int	2	0 to 65535	%u
unsigned short int	1	0 t0 256	%hu
long int	4	-2,147,483,648 to 2,147,483,647	%ld
unsigned long int	4	0 to 4,294,967,295	%lu
long double	10	3.4E -4932 to 1.1E +4932	%Lf

Integer

Integers are whole numbers having a range of values that are supported by a certain computer. In C, there are three types of integer storage: **short int**, **int**, and **long int**, which are available in both signed and unsigned forms. That is integers can store both zero, positive and negative values but no decimal values.

int roll;

int roll, mark;

```
short int id;
signed int x;
```

Short int represents relatively small integer values and takes half the storage space of a typical int number. unsigned int employ all of the bits for the magnitude of the number and are always positive. Because the default declaration implies a signed number, the use of the qualifier signed on integers is optional.

Floating point

Floating-point variables are used to store real numbers with 6 digits of precision. In C, floating-point numbers are defined by the keyword float.To define a number with 14 numbers of precision, the type double can be used. These are referred to as double-precision numbers. long double can be used to increase precision even further. Examples are

```
float rate;
double price;
```

Character

The char data type can be used to define a single character.

```
char demo = 'x';
```

Void

The **void** type has no values. This is typically used to specify the type of function that returns no value to the caller function.

2.User Defined Data Types

The data types that are defined by the user are called the user-defined derived data type.

STRUCTURE

A Structure is used to organize a group of related data items of different data types referring to a single entity. i.e., a single variable capable of holding data items of different data types.

The keyword used to create a structure is a struct. The advantage of using a structure is that the accessibility of members becomes easier since all the members of a specific structure get the allocation of continuous memory and therefore it minimizes the memory access time.

Generally, a structure can be declared as:

```
struct tag_name
{
  data_type1 svar_1;
  data_type2 svar_2;
  ....
  ....
  data_typen svar_n;
};
```

The structure variables can be defined

```
struct tag_name svar_1, svar_2 ...svar_n;
```

Example of a structure

```
struct sample {
  int a, b;
  float c, d;
  char e, f;
```

```
};
struct sample v1, v2, v3; //structure definition
```

UNION

A union is also a collection of different data types in **C** but that allows to store different data types in the same memory location. User can define a union with many members, but only one member can contain a value at any given time. Unions provide an efficient way of using the same memory location for multiple-purpose. A union is same as structures but the difference is that only one member can be accessed at a time because the memory is created only for one member which has the highest number of bytes in size.

A union is declared by using the keyword union and members of the union can be accessed by using dot (.) operator.

The declaration and definition of the union is:

```
union tag_name {
  data_type1 uvar_1;
  data_type2 uvar_2;
  ......
  data_typen uvar_n;
};
union tag_name uvar_1, uvar_2,....uvar_n;
```

Example

```
union sample {
  int age;
```

```
  float price;
  char name;
};
union sample s;
```

TYPEDEF

The keyword typedef is used to create a new name (alias) for an existing data type. It does not create a new data type. The syntax of using typedef is as follows:

```
typedef  existing_type  new_data_type;
```

ENUM

enum is a keyword used to create an enumerated data type. An enum (**enumerated data type**) is a special data type consisting of a set of named values called elements or members. It is mainly used to assign names to integral constants, which makes a program more readable.

The format for creating an enum type is

enum identifier (value1, value2, ... , valueN);

Example

```
enum Boolean {
    false,
    true
};
```

EMPTY DATA TYPE

void:

void keyword is an empty data type that represents **no value**. It is used in **functions** and **pointers**.

3.Derived Data Types

Derived types are data types that are derived from fundamental data types. Arrays, pointers, function types, and so on are examples.

ARRAY

Array in C stores multiple values of the same data type. That means we can have an array of integers, chars, floats, doubles, etc

Syntax

Type variable_name[size];

Example

int number[5];

float height[7];

POINTER

Pointer is a variable that can hold address of another variable

Syntax

type *var-name;

Example

```
int var = 20;

 int* p = &var;
```

Here, 20 is assigned to the var variable. And, the address of var is assigned to the p pointer.

FUNCTION

A functionn is a self contained block of code that performs a particular task.

Symbolic Constants

A symbolic constant is a name given to any constant. In C, the pre-processor directive #define is used for defining symbolic constants. #defineinstructions are usually placed at the beginning of the program. By convention, the names of symbolic constants are written in uppercase, but this is not compulsory. The syntax for creating a symbolic constant is as follows:

```
#define constant_name value
```

For example:

```
#define PI 3.14
```

It will define a symbolic constant PI having value 3.14. When we use PI in our program, it will be replaced with 3.14 by the compiler automatically.

The rules below apply to a #define statement that defines a symbolic constant.

- No blank space is allowed between the # symbol and the word define.
- The character in the line should be #.
- A blank space is required between the constant name and #define and between the constant name and the value.
- A semicolon must not be used at the end of a #define statement.

- Within the program, the symbolic constant should not be assigned any other value.

Example

```
#include <stdio.h>

#define PI 3.1415

int main()

{

float radius=10, area;

area = PI*radius*radius;

printf("Area=%.2f",area);

return 0;

}
```

MODULE 2

C OPERATORS

An **operator** is a symbol that instructs a computer to perform certain operations. Operators are typically used as part of mathematical or logical expressions

Arithmetic Operators

An arithmetic operator performs mathematical operators such as addition, subtraction etc. C supports all basic arithmetic operators are

Operator	Meaning	Example
+	Addition or Unary plus	8 + 2 = 10
–	Subtraction or Unary minus	8 – 2 =6
*	Multiplication	8 * 2 = 16
/	Division	8 / 2 = 4
%	Modulo division	13 % 2 = 1

Relational Operators

The relationship operators are used to check the relationship between two operands. The value of a relational operator is either 1 (true) or 0 (false).

Operator	Meaning	Example
<	is less than	a < b is true.
<=	is less than or equal to	a <= b is true
>	is greater	a > b is false
>=	is grater than an equal to	a >= b is false
==	is equal to	a == b is false

Operator	Meaning	Example
!-	is not equal to	a != b is true

Logical Operators

The logical operators are used to make a decision by testing one or more conditions.

- && – logical AND – True only if all operands are true.
- || – logical OR – True only if either one operand is true.
- ! – logical NOT – True only if the operand is 0

Assignment Operator

Assignment operators are used to assigning values to a variable. The assignment operator in c is =.

Operator	Example	Same as
+=	a += b	a = a + b
-=	a -= b	a = a – b
*=	a *= b	a = a * b
/=	a /= b	a = a / b
%=	a %= b	a = a % b

Increment and Decrement Operator

In C, the operators ++and --are called the increment and decrement operators. The increment operator ++increases the value of

an operand by 1 and the decrement operator --decreases the value of an operand by 1. They both are unary operators.

Bitwise Operator

For manipulation of data at bit level, C supports special operators known as bitwise operators.

Operator	Meaning
&	Bitwise AND
|	Bitwise OR
^	Bitwise exclusive OR
`	Bitwise complement
<<	Shift left
>>	Shift right

Conditional Operator

C has a special ternary operator ?: called conditional operator. The conditional operator behaves similarly to the 'if-else' statement. The syntax

```
condition ? true-statement : false-statement;
```

Here expression1 is a boolean condition that can be either true or false. The conditional operator will return expression2, if the value of expression1 is true. Similarly, if the value of expression1 is false, then the operator will return expression3

```
max = (a > b) ? a : b ;
```

First, the expression a > b is evaluated. If it is true, the statement will be equivalent to max = a;. If a > b is false, then the statement will be equivalent to max = b.

Special Operators

C also support some special operators such as comma operator, sizeof operator, pointer operators, etc.

The Comma Operator

The comma operator is used to link the related expression together. For example: **float** a, b, c;

The sizeof Operator

The sizeof operator return the number of bytes a operand occupies. It can be used to know the size of variables, constants, arrays, structures etc. For example:

```
x = sizeof(mark);
```

Arithmetic Expressions

An arithmetic expression is a combination of variables, constants, and arithmetic operators. Arithmetic operators are

+, -, *, / and %. The operands include integer and floating-type numbers. Some algebraic expressions are

Algebraic Expression	C Expression
(a+b)(a-b)	(a + b) * (a-b)
(ab)/c	(a * b) / c

Arithmetic expressions are evaluated using an assignment statement of the form variable = expression. The expression is evaluated first and the value is assigned to the variable. An example of an evaluation statement is,

c = a - b / d + e

Precedence of Arithmetic Operators in C

To determine the meaning and value of an expression in an unambiguous manner, we apply the operator precedence and associativity rules. Arithmetic expressions without parentheses are evaluated from left to right using the rules of operator precedence. In C, arithmetic operators have two different priority levels.

- Hight priority * / %
- Low priority + -

The basic procedure for evaluating an expression includes two passes from left to right. The high priority operators are applied during the first pass and the low priority operators are applied in the second pass.

For example, the statement x = 8 – 25 / 5 + 2 * 5 – 7 is evaluated as follows,

- **First pass**
  - x = 8 - 5 + 2 * 5 -7
  - x = 8 - 5 + 10 - 7
- **Second pass**
  - x = 3 + 10 - 7
  - x = 13 - 7
  - x = 6

These steps are illustrated in the following figure.

However, parentheses can be used to change the order in which an expression is evaluated. The expression within parentheses assumes the highest priority. The following rules are used for evaluating expressions containing parentheses.

- Parenthesized subexpressions are evaluated from left to right.
- When there are nested parentheses, the evaluation starts with the innermost sub-expression.
- The order of application of operators in evaluating sub-expressions is determined using the precedence rule.
- When two or more operators with the same precedence level exist in a sub-expression, the associativity rule is applied.
- Arithmetic expressions are evaluated from left to right using the rules of precedence.
- The expressions within parentheses assume the highest priority.

Operator Precedence and Associativity in C

The following table shows the complete list of C operators, their precedence levels, and their rules of association.

Operator	Description	Associativity	Rank
()	Function call	Left to right	1

Operator	Description	Associativity	Rank
[]	Array element reference		
+	Unary plus	Right to left	2
–	Unary minus		
++	Increment		
—	Decrement		
!	Logical negation		
~	Ones complement		
*	Pointer reference		
&	Address		
sizeof	Size of an object		
(type)	Type cast (conversion)		
*	Multiplication	Left to right	3

Operator	Description	Associativity	Rank
/	Division		
%	Modulus		
+	Addition	Left to right	4
–	Subtraction		
<<	Left shift	Left to right	5
>>	Right shift		
<	Less than	Left to right	6
<=	Less than or equal to		
>	Greater than		
>=	Greater than or equal to		
==	Equality	Left to right	7
!=	Inequality		

Operator	Description	Associativity	Rank
&	Bitwise AND	Left to right	8
^	Bitwise XOR	Left to right	9
|	Bitwise OR	Left to right	10
&&	Logical AND	Left to right	11
||	Logical OR	Left to right	12
?:	Conditional operator	Right to left	13
=	Assignment operator	Right to left	14
,	Comma operator	Left to right	15

Mathematical Functions

Mathematical functions such as sqrt, cos, sin, log, etc. these functions are supported by the header file is <math.h>

Library Functions

Library functions are built-in functions that are grouped together and placed in a common location called library. All C standard library functions are declared by using many header files. These library functions are created at the time of designing the compilers.

Header File Functions

Some of the header file functions are as follows −

- **stdio.h** − It is a standard i/o header file in which Input/output functions are declared
- **conio.h** − This is a console input/output header file.
- **string.h** − All string related functions are in this header file.
- **stdlib.h** − This file contains common functions which are used in the C programs.
- **math.h** − All functions related to mathematics are in this header file.
- **time.h** − This file contains time and clock related functions. Built functions in stdio.h

MODULE 3

C - Input and Output functions

Input means to feed some data into a program. **Output** means to display some data on screen, printer, or in any file. Some input and output functions are listed below

The getchar() and putchar() Functions

getchar() function reads only single character at a time.

putchar() function puts or print only single character at a time. Example

```
#include <stdio.h>
int main( ) {

  int c;

  printf( "Enter a value :");
  c = getchar( );

  printf( "\nYou entered: ");
  putchar( c );
```

```
  return 0;
}
```

Output

Enter a value : this is test
You entered: t

The gets() and puts() Functions

The **gets()**function reads a string

The **puts()** function writes the string 's'. Example

```
#include <stdio.h>
int main( ) {

  char str[100];

  printf( "Enter a value :");
  gets( str );

  printf( "\nYou entered: ");
  puts( str );

  return 0;
}
```

Output

Enter a value : this is test
You entered: this is test

The scanf() and printf() Functions

The **scanf()** function reads the input from the standard input stream.

The **printf()** function writes the output to the standard output stream. Example

```
#include <stdio.h>
```

```
int main( ) {

   char str[100];
   int i;

   printf( "Enter a value :");
   scanf("%s %d", str, &i);

   printf( "\nYou entered: %s %d ", str, i);

   return 0;
}
```

Output
Enter a value : seven 7
You entered: seven 7

Decision making Statements

Decision making structures require that the programmer specifies one or more conditions to be evaluated or tested by the program, along with a statement or statements to be executed if the condition is determined to be true, and optionally, other statements to be executed if the condition is determined to be false. Decision making statements are

1.if statements
2.switch statements
3.conditional operator statements
4.go to statements

1. <u>if statements</u>

The if statements are in 4 different types, these are
Simple if
If….else
Nested if
Else if ladder

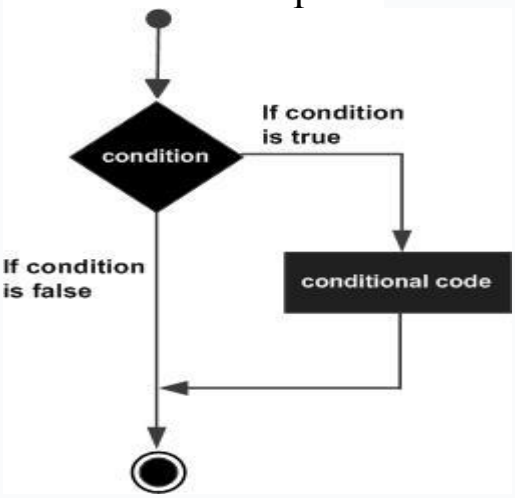
Simple if

An **if** statement consists of a Boolean expression followed by one or more statements.

Syntax

```
 If (test condition)
{
     statements;
}
```

Flowchart of simple if



Example

```
include <stdio.h>
int main () {
   /* local variable definition */
   int a = 10;

   /* check the boolean condition using if statement */

   if( a < 20 ) {
      /* if condition is true then print the following */
      printf("a is less than 20\n" );
   }

   printf("value of a is : %d\n", a);

   return 0;
```

```
}
```

Output

```
a is less than 20;
value of a is : 10
```

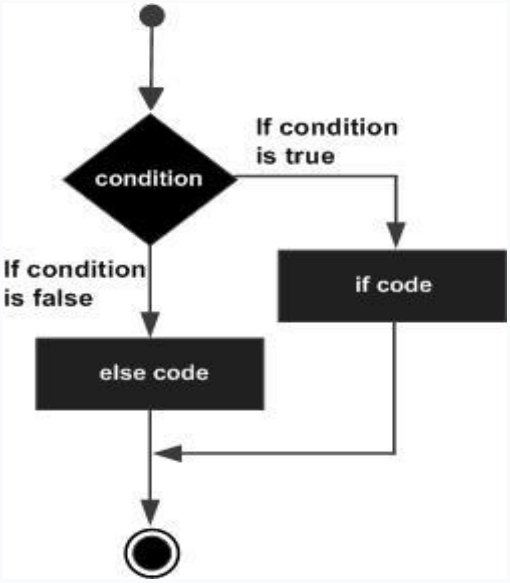
if…..else statements

The if condition is true the true statement is executed else false statement is executed

Syntax

```
If(test condition)
{
  True statements;
}
Else
{
   False statements;
}
```

Flowchart



Example

```
#include <stdio.h>
int main () {
   /* local variable definition */
```

```
  int a = 100;

  /* check the boolean condition */
  if( a < 20 ) {
     /* if condition is true then print the following */
     printf("a is less than 20\n" );
  } else {
     /* if condition is false then print the following */
     printf("a is not less than 20\n" );
  }

  printf("value of a is : %d\n", a);

  return 0;
}
```

Output

```
a is not less than 20;
value of a is : 100
```

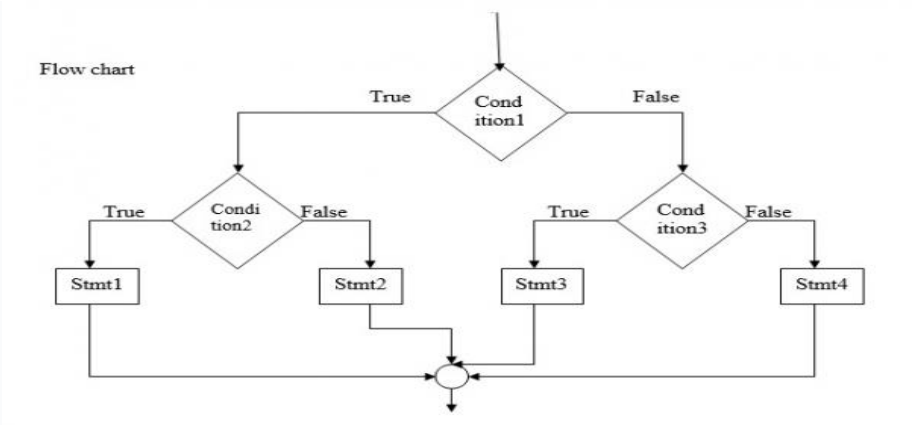
Nested if statements

A 'nested if' is an if statement that is the object of either if (or) an else. 'if' is placed inside another if (or) else.

Syntax

```
if (condition1)
{
  if (condition2)
    stmt1;
  else
    stmt2;
}
else{
  if (condition3)
    stmt3;
  else
    stmt4;
```

```
}
```



Example

```
#include<stdio.h>
void main (){
  int a,b,c,d;
  printf("Enter the values of a,b,c:
");
  scanf("%d,%d,%d",&a,&b,&c);
  if((a>b)&&(a>c)){//Work with 4 numbers//
    if(a>c){
      printf("%d is the largest",a);
    } else {
      printf("%d is the largest",c);
    }
  } else {
    if(b>c){
      printf("%d is the largest",b);
    } else {
      printf("%d is the largest",c);
    }
  }
}
```

Output

```
Enter the values of a,b,c: 3,5,8
```

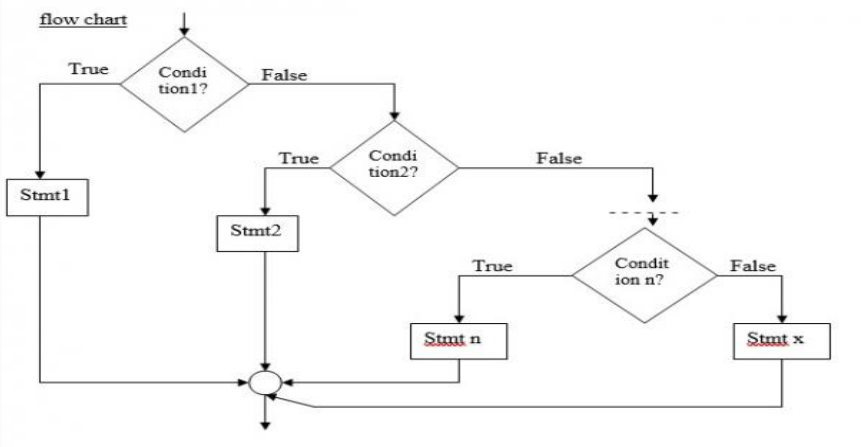
8 is the largest

Else if ladder

This is the most general way of writing a multi-way decision.

Syntax

```
if (condition1)
      stmt1;
else if (condition2)
     stmt2;
    - - - - -
    - - - - -
else if (condition n)
    stmtn;
else
  stmt x;
```



Example

```
#include<stdio.h>
void main (){
  int a,b,c,d;
  printf("Enter the values of a,b,c,d: ");
  scanf("%d%d%d%d",&a,&b,&c,&d);
  if(a>b && a>c && a>d){
```

```
    printf("%d is the largest",a);
  }else if(b>c && b>a && b>d){
    printf("%d is the largest",b);
  }else if(c>d && c>a && c>b){
    printf("%d is the largest",c);
  }else{
    printf("%d is the largest",d);
  }
}
```

Output

Run 1:Enter the values of a,b,c,d: 2 4 6 8

8 is the largest

Run 2: Enter the values of a,b,c,d: 23 12 56 23

56 s the larges

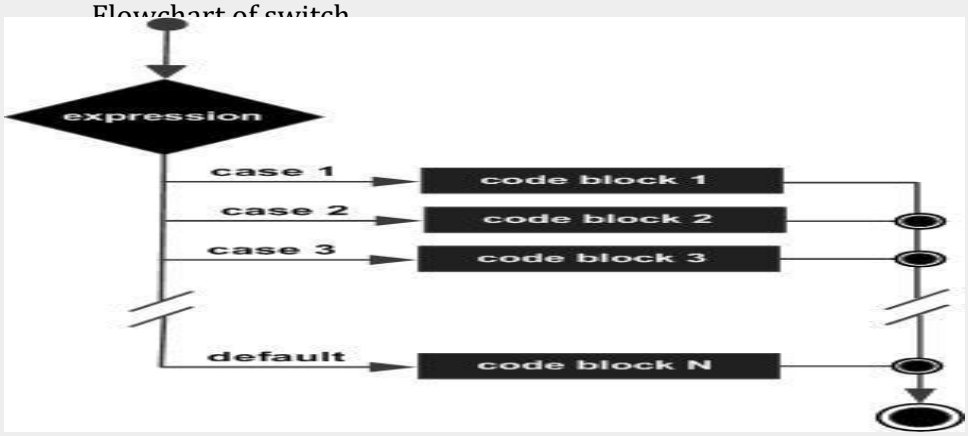
2.Switch statements

A **switch** statement allows a variable to be tested for equality against a list of values. Each value is called a case, and the variable being switched on is checked for each **switch case**.

Syntax

```
switch(expression)
{
  case constant-expression  :   statement(s);
    break; /* optional */

  case constant-expression  :  statement(s);
    break; /* optional */

  /* you can have any number of case statements */
  default : /* Optional */   statement(s);

}
```

Flowchart of switch



Example

```
include <stdio.h>
int main () {

   /* local variable definition */
   char grade = 'C';

   switch(grade) {
      case 'A' :
         printf("Excellent!\n" );
         break;
      case 'B' : printf("Good!\n" );
         break;
      case 'C' :
         printf("Well done\n" );
         break;
      case 'D' :
         printf("You passed\n" );
         break;
      case 'F' :
         printf("Better try again\n" );
         break;
      default :
         printf("Invalid grade\n" );
   }

   printf("Your grade is  %c\n", grade );

   return 0;
}
```

output

```
Well done
Your grade is C
```

3.The ? : Operator

We have covered **conditional operator ? :** in the previous chapter which can be used to replace **if...else** statements.it is also known as ternary operators It has the following general form −

Exp1 ? Exp2 : Exp3;

Where Exp1, Exp2, and Exp3 are expressions. Notice the use and placement of the colon.

The value of a ? expression is determined like this −

- Exp1 is evaluated. If it is true, then Exp2 is evaluated and becomes the value of the entire ? expression.
- If Exp1 is false, then Exp3 is evaluated and its value becomes the value of the expression.

Example

```
#include <stdio.h>
int main()
{
   int age;  // variable declaration
   printf("Enter your age");
   scanf("%d",&age);   // taking user input for age variable
   (age>=18)? (printf("eligible for voting")) : (printf("not eligible for voti
ng"));
   return 0;
}
```

4.Go to statements

A **goto** statement in C programming provides an unconditional jump from the 'goto' to a labeled statement in the same function.

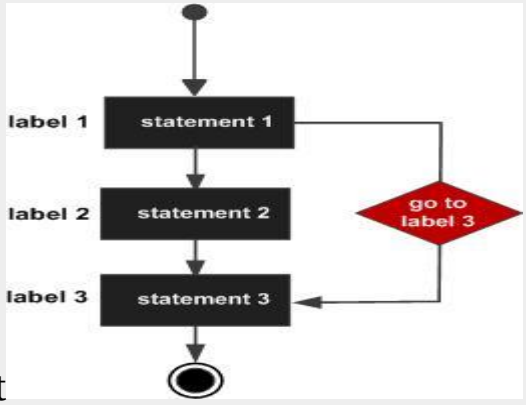
Syntax

goto label;

..

.

label: statement;

flowchart 

example

```
#include <stdio.h>
int main () {

   /* local variable definition */
   int a = 10;

   /* do loop execution */
   LOOP:do {

      if( a == 15) {
         /* skip the iteration */
         a = a + 1;
         goto LOOP;
      }

      printf("value of a: %d\n", a);
      a++;

   }while( a < 20 );

   return 0;
}
```

Output
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 16
value of a: 17
value of a: 18
value of a: 19

Loops in c

Loop means when a block of code needs to be executed several number of times. In general, statements are executed sequentially. A loop statement allows us to execute a statement or group of statements multiple times. Loops are mainly divided into two types entry controlled loop and exit controlled loop.

In **entry controlled loop** first condition is executed and it is true the body of the loop is executed. While and for loops are entry controlled loop

In **exit controlled loop** first body of the loop is executed then the condition will be executed. If condition is false the body of the loop is executed only once. Do..while loop is an entry controlled loop

Loops are

2. While loop
3. For loop
4. Do….while loop

While loop

A **while** loop in C repeatedly executes a target statement as long as a given condition is true.

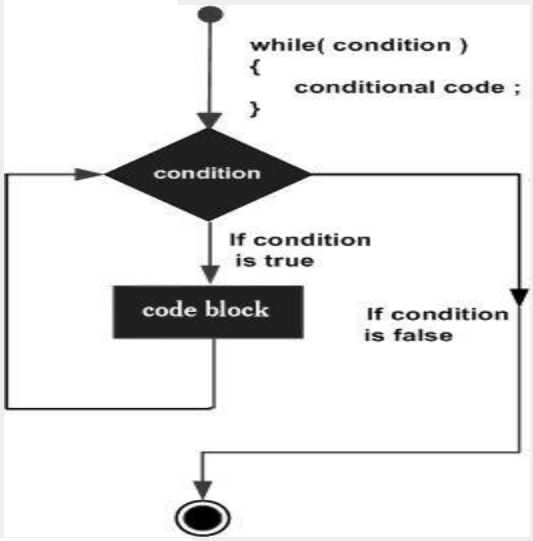
Syntax

while(condition)

```
{
  statement(s);
}
```

When the condition become true the statement (s)will be executed. When the condition becomes false, the program control passes to the line immediately following the loop.

Flowchart



Example

```
#include <stdio.h>
int main () {
  /* local variable definition */
  int a = 10;

  /* while loop execution */
  while( a < 20 ) {
    printf("value of a: %d\n", a);
    a++;
  }

  return 0;
}
```

Output

```
value of a: 10
```

```
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
value of a: 16
value of a: 17
value of a: 18
value of a: 19
```

For loop

A **for** loop is a repetition control structure that allows you to efficiently write a loop that needs to execute a specific number of times.

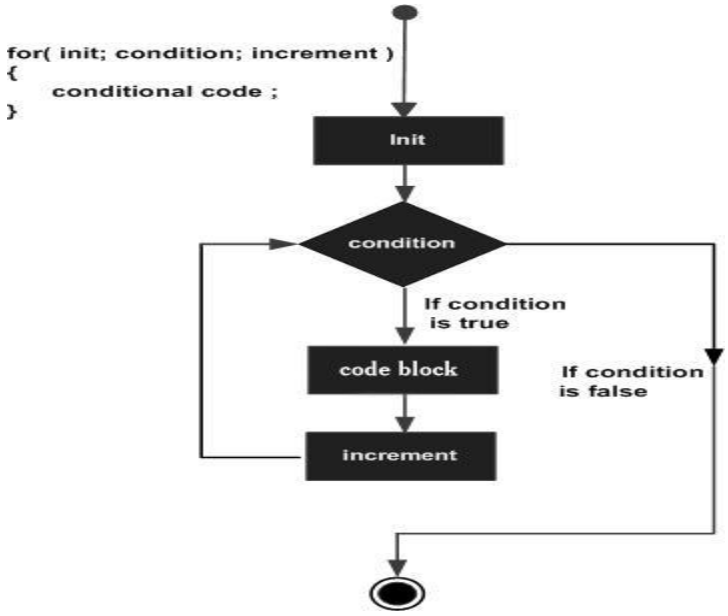
Syntax

```
for ( initialization; condition; increment/decre )
{
  statement(s);
}
```

The **initialization** step is executed first, and only once. This step allows you to declare and initialize any loop control variables.

The **condition** is evaluated. If it is true, the body of the loop is executed. If it is false, the body of the loop does not execute and the flow of control jumps to the next statement just after the 'for' loop. Then incre/decre variable again the process repeated.

Flowchart



Do….while loop

A **do...while** loop is similar to a while loop, except the fact that it is guaranteed to execute at least one time.

Syntax

```
do {
   statement(s);
} while( condition );
```

Notice that the conditional expression appears at the end of the loop, so the statement(s) in the loop executes once before the condition is tested.

If the condition is true, the flow of control jumps back up to do, and the statement(s) in the loop executes again. This process repeats until the given condition becomes false.

Example

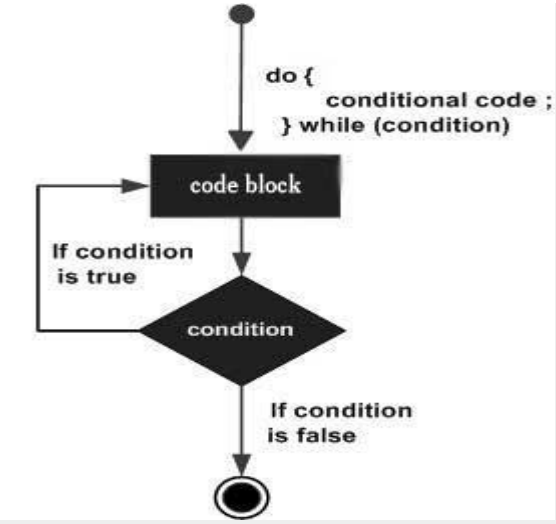
```
#include <stdio.h>
int main () {
   int a;

   /* for loop execution */
   for( a = 10; a < 20; a = a + 1 ){
      printf("value of a: %d\n", a);
   }

   return 0;
```

```
}
```

output

```
value of a: 10
value of a: 11
value of a: 12
value of a: 13
value of a: 14
value of a: 15
value of a: 16
value of a: 17
value of a: 18
value of a: 19
```

Flowchart



Nested loop

A loop inside another loop is called nested loop

The syntax for a **nested for loop** is

```
for ( init; condition; increment ) {

  for ( init; condition; increment ) {
    statement(s);
  }
  statement(s);
}
```

The syntax for a **nested while loop** is

```
while(condition) {

  while(condition) {
    statement(s);
  }
  statement(s);
}
```

The syntax for a **nested do...while loop** is

```
do {
  statement(s);

  do {
    statement(s);
  }while( condition );

}while( condition );
```

Jumps in loops

Jump statements alter the normal execution path of a program. Jump statements are used when we want to skip some statements inside loop or terminate the loop immediately when some condition becomes true.

There are 4 types of Jump statements in C language.

1. Break Statement
2. Continue Statement
3. Go to Statement
4. Return Statement.

break

- It is a keyword which is used to terminate the loop (or) exit from the block.
- The control jumps to next statement after the loop (or) block.

- break is used with for, while, do-while and switch statement.
- When break is used in nested loops then, only the innermost loop is terminated.

Syntax

```
{       Stmt1;
        Stmt2;
        break;
        Stmt3;
        Stmt4;
}
```

Example

```
#include<stdio.h>
main( ){
  int i;
  for (i=1; i<=5; i++){
    printf ("%d", i);
    if (i==3)
    break;
  }
}
```

Output

```
1 2 3
```

Continue

The continue statement skips the current iteration of the loop and continues with the next iteration.

Syntax

```
{
        Stmt1;
        Stmt2;
        continue;
        Stmt3;
        Stmt4;
}
```

Example

```
#include<stdio.h>
main( ){
  int i;
  for (i=1; i<=5; i++){
    if (i==2)
    continue;
    printf("%d", i)
  }
}
```

Output

```
1 2 3 4 5
```

Return

It terminates the execution of function and returns the control of calling function

The syntax

```
        return[expression/value];
```

Example

```
#include<stdio.h>
main(){
```

```
  int a,b,c;
  printf("enter a and b value:");
  scanf("%d%d",&a,&b);
  c=a*b;
  return(c);
}
```

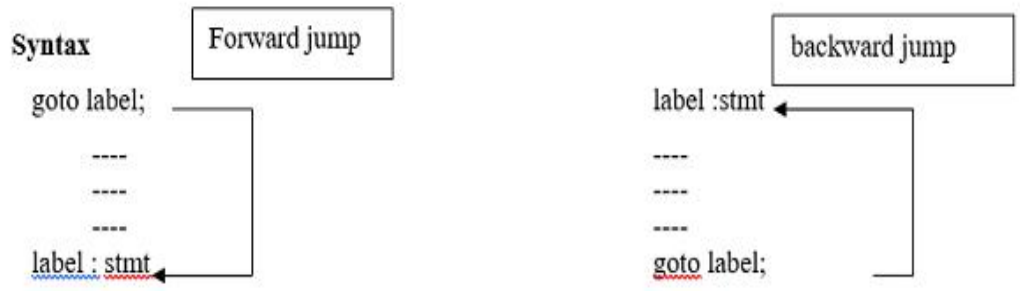
Output

```
enter a and b value:2 4
Process returned 8 (0x8)
```

goto

It is used after the normal sequence of program execution by transferring the control to some other part of program.

The syntax



Example

```
#include<stdio.h>
main( ) {
  printf("Hello");
  goto l1;
  printf("How are");
  l1: printf("you");
}
```

Output

```
Hello you
```

MODULE 4

ARRAY

Array is a kind of data structure that can store a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data having same type.

Declaring Arrays

```
type arrayName [ arraySize ];
```

This is called a *single-dimensional array or one dimensional array*. The **arraySize** must be an integer constant greater than zero and **type** can be any valid C data type.

For example, to declare a 10-element array called **balance** of type double, use this statement −

```
double balance[10];
```

Initializing Arrays

double balance[5] = {1000.0, 2.0, 3.4, 7.0, 50.0};

	0	1	2	3	4
balance	1000.0	2.0	3.4	7.0	50.0

Accessing Array Elements

An element is accessed by indexing the array name. This is done by placing the index of the element within square brackets after the name of the array. For example

```
balance[4] = 50.0;
```

Example

```
#include <stdio.h>
```

```
int main () {

   int n[ 10 ]; /* n is an array of 10 integers */
   int i,j;

   /* initialize elements of array n to 0 */
   for ( i = 0; i < 10; i++ ) {
      n[ i ] = i + 100; /* set element at location i to i + 100 */
   }

   /* output each array element's value */
   for (j = 0; j < 10; j++ ) {
      printf("Element[%d] = %d\n", j, n[j] );
   }

   return 0;
}
```

Output

```
Element[0] = 100
Element[1] = 101
Element[2] = 102
Element[3] = 103
Element[4] = 104
Element[5] = 105
Element[6] = 106
Element[7] = 107
Element[8] = 108
Element[9] = 109
```

Two-dimensional Arrays

A two-dimensional array is contain two dimensions. To declare a two-dimensional integer array of size [x][y], you would write something as follows −

```
type arrayName [ x ][ y ];
```

Where **type** can be any valid C data type and **arrayName** will be a valid C identifier. A two-dimensional array can be considered as a table which will have x number of rows and y number of columns. A two-dimensional array **a**, which contains three rows and four columns can be shown as

int a[3][4];

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Thus, every element in the array **a** is identified by an element name of the form **a[i][j]**, where 'a' is the name of the array, and 'i' and 'j' are the subscripts that uniquely identify each element in 'a'.

Initializing Two-Dimensional Arrays

Two dimensional arrays may be initialized by specifying bracketed values for each row. Following is an array with 3 rows and each row has 4 columns.

```
int a[3][4] = {
   {0, 1, 2, 3} ,   /*  initializers for row indexed by 0 */
   {4, 5, 6, 7} ,   /*  initializers for row indexed by 1 */
   {8, 9, 10, 11}   /*  initializers for row indexed by 2 */
};
```

The nested braces, which indicate the intended row, are optional. The following initialization is equivalent to the previous example −

```
int a[3][4] = {0,1,2,3,4,5,6,7,8,9,10,11};
```

Accessing Two-Dimensional Array Elements

An element in a two-dimensional array is accessed by using the subscripts, i.e., row index and column index of the array. For example

```
int val = a[2][3];
```

Example

```
#include <stdio.h>
int main () {

   /* an array with 5 rows and 2 columns*/
   int a[5][2] = { {0,0}, {1,2}, {2,4}, {3,6},{4,8}};
   int i, j;
```

```
  /* output each array element's value */
  for ( i = 0; i < 5; i++ ) {

    for ( j = 0; j < 2; j++ ) {
      printf("a[%d][%d] = %d\n", i,j, a[i][j] );
    }
  }

  return 0;
}
```

Output

```
a[0][0]: 0
a[0][1]: 0
a[1][0]: 1
a[1][1]: 2
a[2][0]: 2
a[2][1]: 4
a[3][0]: 3
a[3][1]: 6
a[4][0]: 4
a[4][1]: 8
```

Multi dimensional Arrays

Multidimensional arrays contains three or more dimension. Here is the general form of a multidimensional array declaration −

type name[size1][size2]...[sizeN];

For example, the following declaration creates a three dimensional integer array −

```
int threedim[5][10][4];
```

Strings

Strings are actually one-dimensional array of characters terminated by a **null** character '\0'. Sequence of characters are called string

The following declaration and initialization create a string consisting of the word "Hello". To hold the null character at the end of the array, the size of the character array containing the string is one more than the number of characters in the word "Hello."

```
char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
char greeting[] = "Hello";
```

Following is the memory presentation of the above defined string

Index	0	1	2	3	4	5
Variable	H	e	l	l	o	\0
Address	0x23451	0x23452	0x23453	0x23454	0x23455	0x23456

```
#include <stdio.h>
int main () {

  char greeting[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
  printf("Greeting message: %s\n", greeting );
  return 0;
}
```

Output: Greeting message: Hello

String manipulation functions

	Function & Purpose
1	**strcpy(s1, s2);** Copies string s2 into string s1.
2	**strcat(s1, s2);** Concatenates string s2 onto the end of string s1.
3	**strlen(s1);** Returns the length of string s1.
4	**strcmp(s1, s2);**

	Returns 0 if s1 and s2 are the same; less than 0 if s1<s2; greater than 0 if s1>s2.
5	**strchr(s1, ch);** Returns a pointer to the first occurrence of character ch in string s1.
6	**strstr(s1, s2);** Returns a pointer to the first occurrence of string s2 in string s1.

```c
#include <stdio.h>
#include <string.h>
int main () {
   char str1[12] = "Hello";
   char str2[12] = "World";
   char str3[12];
   int  len ;

   /* copy str1 into str3 */
   strcpy(str3, str1);
   printf("strcpy( str3, str1) :  %s\n", str3 );

   /* concatenates str1 and str2 */
   strcat( str1, str2);
   printf("strcat( str1, str2):   %s\n", str1 );

   /* total lenghth of str1 after concatenation */
   len = strlen(str1);
   printf("strlen(str1) :  %d\n", len );

   return 0;
}
```

output

```
strcpy( str3, str1) :  Hello
strcat( str1, str2):   HelloWorld
strlen(str1) :  10
```

User defined function

A function is a block of code that performs a specific task. Every C program has at least one function, which is **main()**, User can defined functions to their needs are known as user defined function.

Each function performs a specific task.

Function declaration

Function definition
Function calling
A function **declaration** tells the compiler about a function's name, return type, and parameters. A function **definition** provides the actual body of the function.

Defining a Function

The general form of a function definition in C programming language is as follows −

```
return_type function_name (parameter list)
{
  body of the function
}
```

A function definition in C programming consists of a *function header* and a *function body*. Here are all the parts of a function −

- **Return Type** − A function may return a value. The **return_type** is the data type of the value the function returns. Some functions have without returning a value, the return_type is the keyword **void**.
- **Function Name** − This is the actual name of the function. The function name and the parameter list together constitute the function signature.
- **Parameters** − When a function is invoked, you pass a value to the parameter. This value is referred to as actual parameter or argument. Parameters are optional; that is, a function may contain no parameters.
- **Function Body** − The function body contains a collection of statements that define what the function does.

Function Declarations

A function **declaration** tells the compiler about a function name and how to call the function. The actual body of the function can be defined separately.

A function declaration has the following parts −

```
return_type function_name( parameter list );
```

For the above defined function max(), the function declaration is as follows −

```
int max(int num1, int num2);
```

Parameter names are not important in function declaration only their type is required, so the following is also a valid declaration −

```
int max(int, int);
```

Function declaration is required when you define a function in one source file and you call that function in another file. In such case, you should declare the function at the top of the file calling the function.

Calling a Function

When a program calls a function, the program control is transferred to the called function. A called function performs a defined task and when its return statement is executed or when its function-ending closing brace is reached, it returns the program control back to the main program.

For example −

```
#include <stdio.h>
/* function declaration */
int max(int num1, int num2);

int main () {

   /* local variable definition */
   int a = 100;
   int b = 200;
   int ret;

   /* calling a function to get max value */
   ret = max(a, b);

   printf( "Max value is : %d\n", ret );

   return 0;
}

/* function returning the max between two numbers */
int max(int num1, int num2) {

   /* local variable declaration */
   int result;
```

```
  if (num1 > num2)
    result = num1;
  else
    result = num2;

  return result;
}
```

Function Arguments

If a function is to use arguments, it must declare variables that accept the values of the arguments. These variables are called the **formal parameters** of the function.

Formal parameters behave like other local variables inside the function and are created upon entry into the function and destroyed upon exit.

While calling a function, there are two ways in which arguments can be passed to a function −

The **call by value** method of passing arguments to a function copies the actual value of an argument into the formal parameter of the function.

Consider the function **swap()** definition as follows.

```
/* function definition to swap the values */
void swap(int x, int y) {

  int temp;

  temp = x; /* save the value of x */
  x = y;    /* put y into x */
  y = temp; /* put temp into y */

  return;
}
```

The **call by reference** method of passing arguments to a function copies the address of an argument into the formal parameter. Inside the function, the address is used to access the actual argument used in the call.

you need to declare the function parameters as pointer types as in the following function **swap()**, which exchanges the values of the two integer variables pointed to, by their arguments.

```
/* function definition to swap the values */
void swap(int *x, int *y) {
```

```
  int temp;
  temp = *x;    /* save the value at address x */
  *x = *y;      /* put y into x */
  *y = temp;    /* put temp into y */

  return;
}
```

Types of User-defined Functions in C

1. Function with no arguments and no return value
2. Function with no arguments and a return value
3. Function with arguments and no return value
4. Function with arguments and with return value

1. Function with No Arguments and No Return Value

Functions that have no arguments and no return values.

Syntax:

```
// void return type with no arguments
void function_name()
{
    // no return value
    return;
}
```

2. Function with No Arguments and With Return Value

Functions that have no arguments but have some return values. Such functions are used to perform specific operations and return their value.

Syntax:

```
return_type function_name()
{
   // program
```

```
  return value;
}
```

3. Function With Arguments and No Return Value

Functions that have arguments but no return values. Such functions are used to display or perform some operations on given arguments.

Syntax:

```
void function_name(type1 argument1, type2 argument2,...typeN argumentN)
{
   // Program
   return;
}
```

4. Function With Arguments and With Return Value

Functions that have arguments and some return value. These functions are used to perform specific operations on the given arguments and return their values to the user.

Syntax:

```
return_type function_name(type1 argument1, type2 argument2,...typeN argumentN)
{
   // program
   return value;
}
```

Recursion

Recursion is the process of repeating items in a self-similar way. A function call itself is known is called recursion.

The following example calculates the factorial of a given number using a recursive function.

```
#include <stdio.h>

 int factorial(unsigned int i) {

  if(i <= 1) {
    return 1;
  }
  return i * factorial(i - 1);
}

int  main() {
  int i = 12;
  printf("Factorial of %d is %d\n", i, factorial(i));
  return 0;
}
```

Output
Factorial of 12 is 479001600

Scope of a variable

1. Local Variables

When we declare variables inside a function or an inner block of code, then these variables are known as local variables. They can only be used inside the *function scope* or the *block scope*.

```
#include <stdio.h>

int main() {
  int sum = 0;

  // inner block of code having block scope
  {
    int a = 5;
    int b = 10;
  }
  / this statement will throw an error because a and b are not available outside the above
block of code

  sum = a + b;

  return 0;
}
```

2. Global Variables

When we declare variables outside of all the functions then these variables are known as global variables. These variables always have *file scope* and can be *accessed* anywhere in the program, they also remain in the memory until the execution of our program finishes.

```
#include <stdio.h>

// global variable
int side = 10;

int main() {
  int squareArea = side * side;

  printf("Area of Square : %d", squareArea);

  return 0;
}
```

Lifetime of a variable is the *time* for which the variable is taking up a *valid space* in the system's memory, it is of types:

- *static variable: The value of static variables persists until the end of the program.it can use the keyword is static*
- *Dynamic variable: The value of dynamic variables can change during the run time of a program*
- *Automatic variables: It is declared inside a function.it uses the keyword is auto*
- *External variables: It is declared outside the function*

MODULE 5

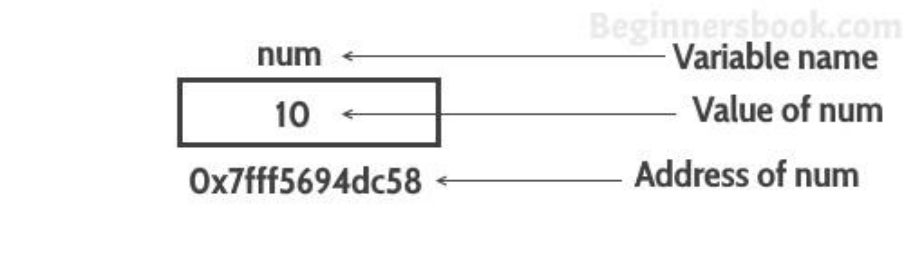
Pointers

Pointer is a variable that can hold address of another variable. It is a variable whose value is the address of another variable, i.e., direct address of the memory location. Like any variable or constant, you must declare a pointer before using it to store any variable

address. Accessing the address of a variable can be done with the help of the operator &.

Syntax

type *var-name;



Example

```
#include <stdio.h>

int main () {

   int  var = 20;   /* actual variable declaration */
   int  *ip;        /* pointer variable declaration */

   ip = &var;  /* store address of var in pointer variable*/

   printf("Address of var variable: %x\n", &var  );

   /* address stored in pointer variable */
   printf("Address stored in ip variable: %x\n", ip );

   /* access the value using the pointer */
   printf("Value of *ip variable: %d\n", *ip );

   return 0;
}
```

Output

```
Address of var variable: bffd8b3c
Address stored in ip variable: bffd8b3c
Value of *ip variable: 20
```

Files

A file represents a sequence of bytes, regardless of it being a text file or a binary file. C programming language provides access on high level functions as well as low level (OS level) calls to handle file on your storage devices

Opening Files

You can use the **fopen( )** function to create a new file or to open an existing file. This call will initialize an object of the type **FILE**, which contains all the information necessary to control the stream. The prototype of this function call is as follows −

```
FILE *fopen( const char * filename, const char * mode );
```

Here, filename is a string literal, which you will use to name your file, and access mode can have one of the following values −

	Mode & Description
1	**r** Opens an existing text file for reading purpose.
2	**w** Opens a text file for writing. If it does not exist, then a new file is created. Here your program will start writing content from the beginning of the file.
3	**a** Opens a text file for writing in appending mode. If it does not exist, then a new file is created. Here your program will start appending content in the existing file content.
4	**r+** Opens a text file for both reading and writing.
5	**w+** Opens a text file for both reading and writing. It first truncates the file to zero length if it exists, otherwise creates a file if it does not exist.

6	**a+** Opens a text file for both reading and writing. It creates the file if it does not exist. The reading will start from the beginning but writing can only be appended.

Closing a File

To close a file, use the fclose( ) function. The prototype of this function is −

```
int fclose( FILE *fp );
```

The **fclose(-)** function returns zero on success. This function actually flushes any data still pending in the buffer to the file, closes the file, and releases any memory used for the file.

There are various functions provided by C standard library to read and write a file, character by character, or in the form of a fixed length string.

Writing a File

Following is the simplest function to write individual characters to a stream −

```
int fputc( int c, FILE *fp );
```

The function **fputc()** writes the character value of the argument c to the output stream referenced by fp. It returns the written character written on success otherwise **EOF** if there is an error. You can use the following functions to write a null-terminated string to a stream −

```
int fputs( const char *s, FILE *fp );
```

The function **fputs()** writes the string **s** to the output stream referenced by fp. It returns a non-negative value on success, otherwise **EOF** is returned in case of any error. You can use **int fprintf(FILE *fp,const char *format, ...)** function as well to write a string into a file. Try the following example.

```
#include <stdio.h>

main() {
```

```
  FILE *fp;

  fp = fopen("/tmp/test.txt", "w+");
  fprintf(fp, "This is testing for fprintf...\n");
  fputs("This is testing for fputs...\n", fp);
  fclose(fp);
}
```

Reading a File

Given below is the simplest function to read a single character from a file −

```
int fgetc( FILE * fp );
```

The **fgetc()** function reads a character from the input file referenced by fp. The return value is the character read, or in case of any error, it returns **EOF**. The following function allows to read a string from a stream −

```
char *fgets( char *buf, int n, FILE *fp );
```

The functions **fgets()** reads up to n-1 characters from the input stream

If this function encounters a newline character '\n' or the end of the file EOF before they have read the maximum number of characters, then it returns only the characters read up to that point including the new line character. You can also use **int fscanf(FILE *fp, const char *format, ...)** function to read strings from a file, but it stops reading after encountering the first space character.

```
#include <stdio.h>

main() {

   FILE *fp;
   char buff[255];

   fp = fopen("/tmp/test.txt", "r");
   fscanf(fp, "%s", buff);
   printf("1 : %s\n", buff );

   fgets(buff, 255, (FILE*)fp);
   printf("2: %s\n", buff );
```

```
  fgets(buff, 255, (FILE*)fp);
  printf("3: %s\n", buff );
  fclose(fp);

}
```

Output

```
1 : This
2: is testing for fprintf...

3: This is testing for fputs...
```

Let's see a little more in detail about what happened here. First, **fscanf()** read just **This** because after that, it encountered a space, second call is for **fgets()** which reads the remaining line till it encountered end of line. Finally, the last call **fgets()** reads the second line completely.

THANK YOU